

a) Input subject

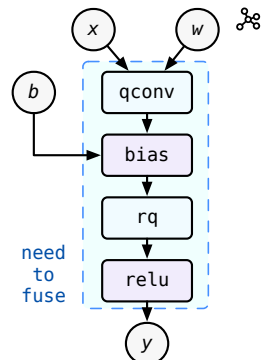
mod extension

use base

use ir

```
fn subject(
  x: tensor<i8, [1,1,1,2]>,
  w: tensor<i8, [1,1,2,1]>,
  b: tensor<i32, [1]>
) -> tensor<i8, [1,1,1,1]> {
  [layout: "NHWC", kernel_layout: "HWIO"]
  let conv = qconv(x, w)
  let biased = bias(conv, b)
  [acc_scale: 0.25, out_scale: 0.5, output_zero: 0]
  let quantized = requantize(biased)
  [output_zero: 0]
  let output = relu(quantized)
  return output
}
```

f



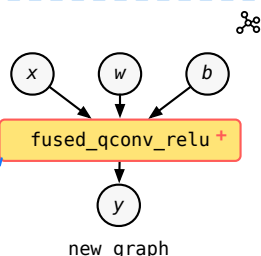
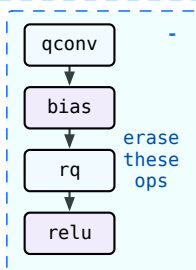
b) Check graph edit

local fn fuse(m: Mod, relu: Op) -> bool

```
# replacement excerpt
let old = ir.outs(relu)[0]
let inp = ir.args(conv)
+ [ir.args(bias)[1]]
let res = ir.call(m, relu,
  ir.find(m,
    "fused_qconv_relu"),
  inp, ir.type(old))
```

copy attrs: conv + requantize

f



c) Output subject

```
fn subject(
  x: tensor<i8, [1,1,1,2]>,
  w: tensor<i8, [1,1,2,1]>,
  b: tensor<i32, [1]>
) -> tensor<i8, [1,1,1,1]> {
  [acc_scale: 0.25, kernel_layout: "HWIO",
  layout: "NHWC", out_scale: 0.5, output_zero: 0]
  let fused16: tensor<i8, [1,1,1,1]> =
    fused_qconv_relu(x,w,b)
  return fused16
}
```

f

